리눅스(Linux) 메뉴얼



Ubuntu 24.04 버전

2026-01-12

관리자: 김건일 박사과정

목차

1. 윈도우 기능 및 환경변수 설정
2. WSL2 설치 방법
3. 리눅스(버전: Ubuntu 24.04) 실행
4. 리눅스 새로운 계정 생성 방법

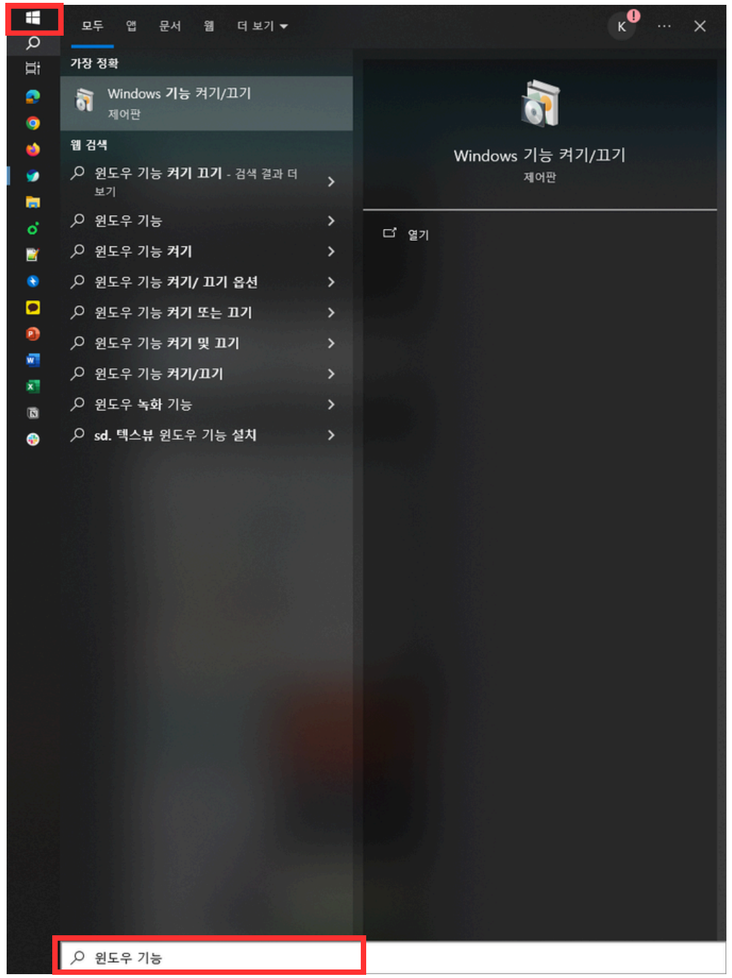
5.1 (옵션) 우분투 서버 접속 프로그램 설치 (Win 10/11 전용)
5.2 (옵션) 우분투 서버 접속 프로그램 설치 (Mac 전용)

6.1 (andlab 계정 기준) Conda 가상환경 생성 방법
6.2 (andlab 계정 기준) Conda 패키지 설치 방법

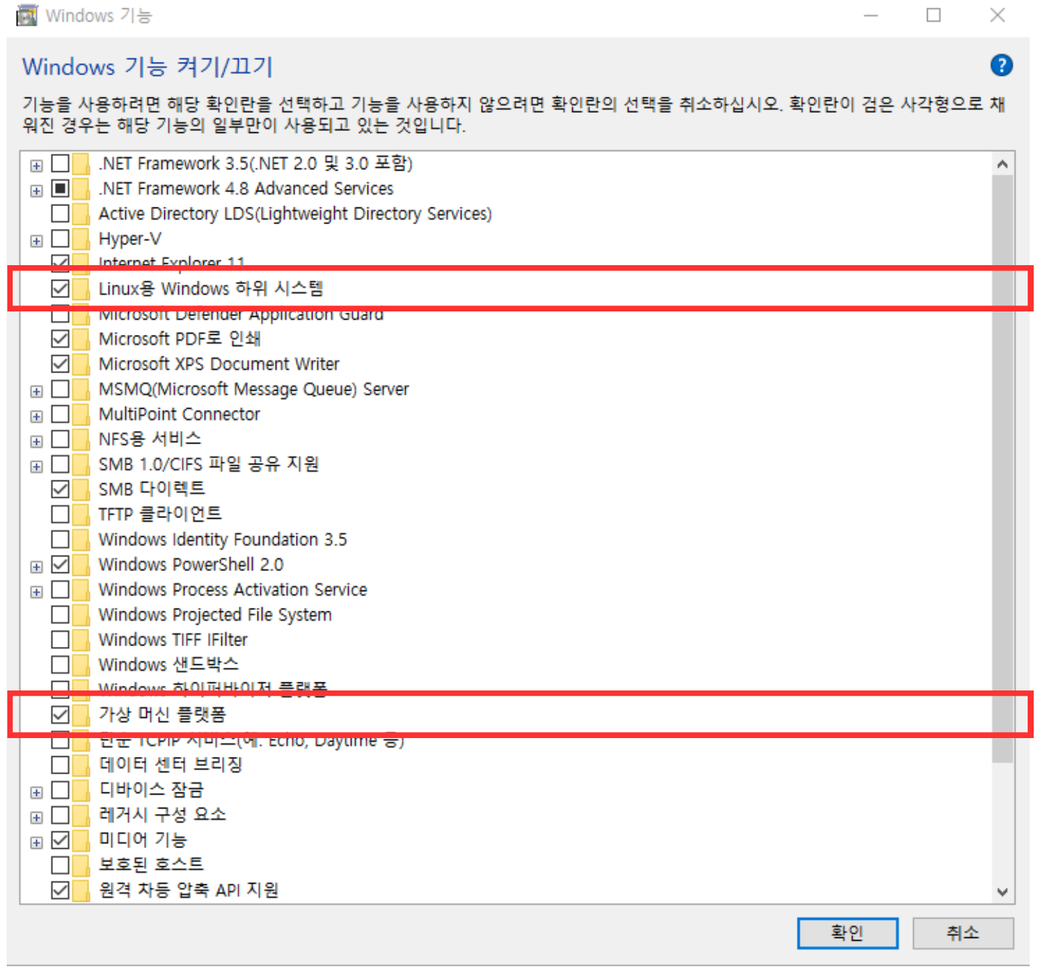
7.1 (개인 계정 기준) CUDA / CUDNN 설정
7.2 (개인 계정 기준) Anaconda 설치

8. (모든 계정 가능) Jupyter-Notebook 설치 및 실행 방법
9. (모든 계정 가능) VS-Code 설치 및 실행 방법 [Version 1 & 2]
10. (모든 계정 가능) 전체 디스크 확인 방법
11. (모든 계정 가능) 공용 데이터셋 및 모델

1. 윈도우 기능 및 환경변수 설정



1. 윈도우 키 클릭
2. “윈도우 기능“ 입력 및 클릭

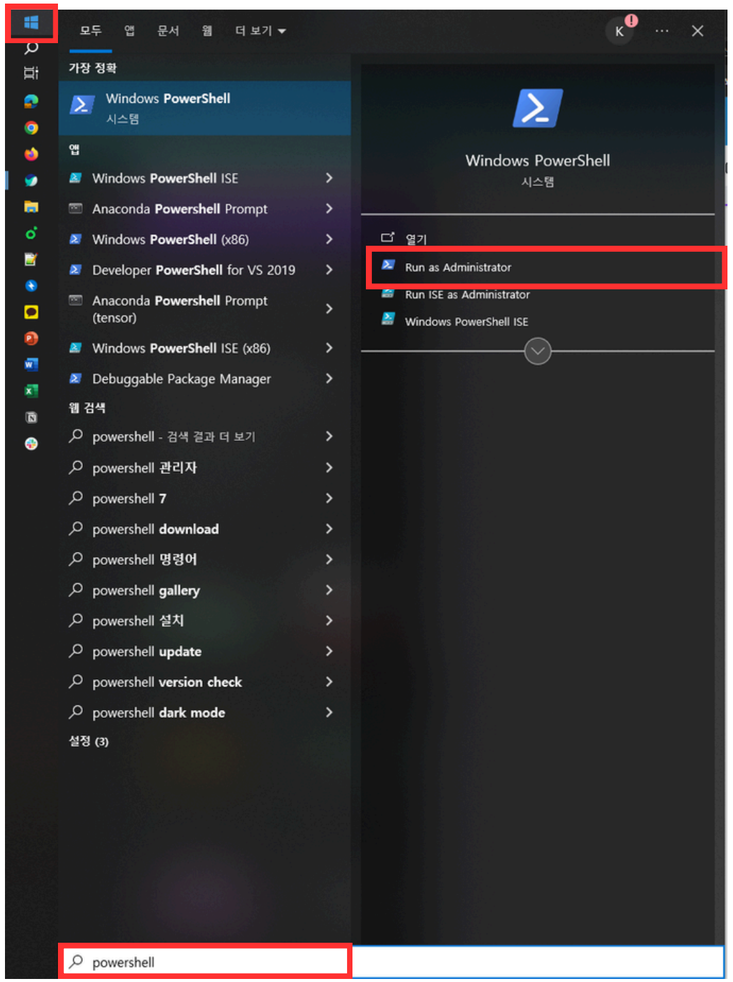


3. Linux용 Windows 하위 시스템 채크
4. 가상 머신 플랫폼 채크

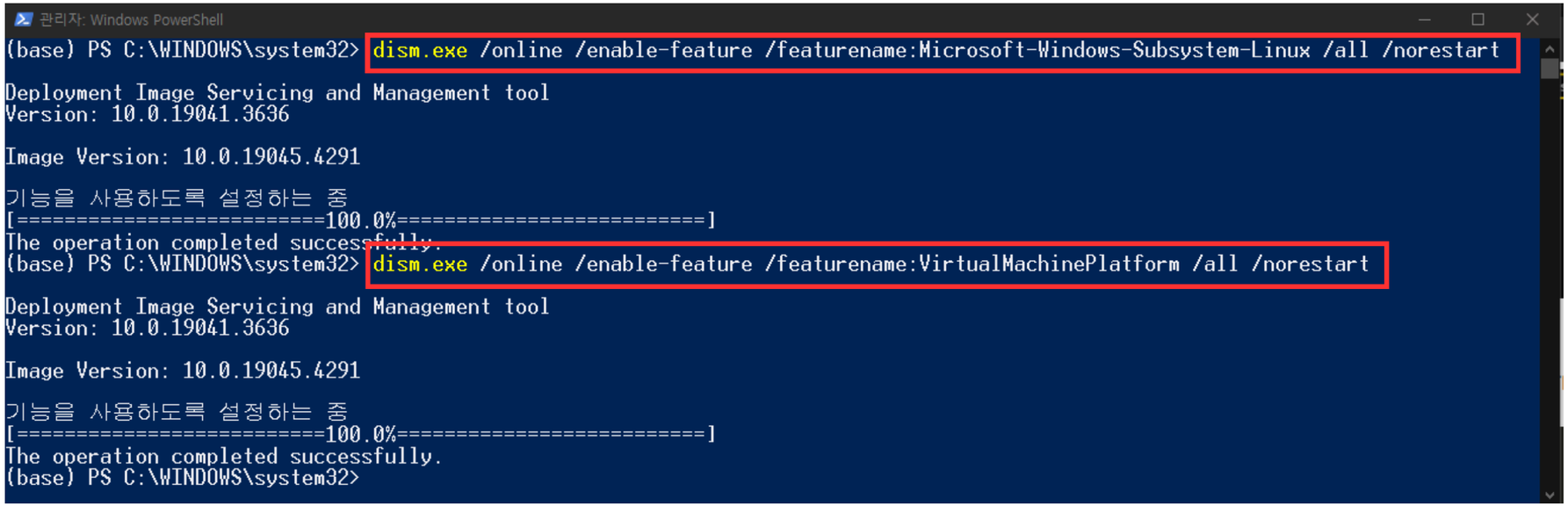
컴퓨터 다시 시작

5. 컴퓨터 다시 시작

2. WSL2 설치 방법



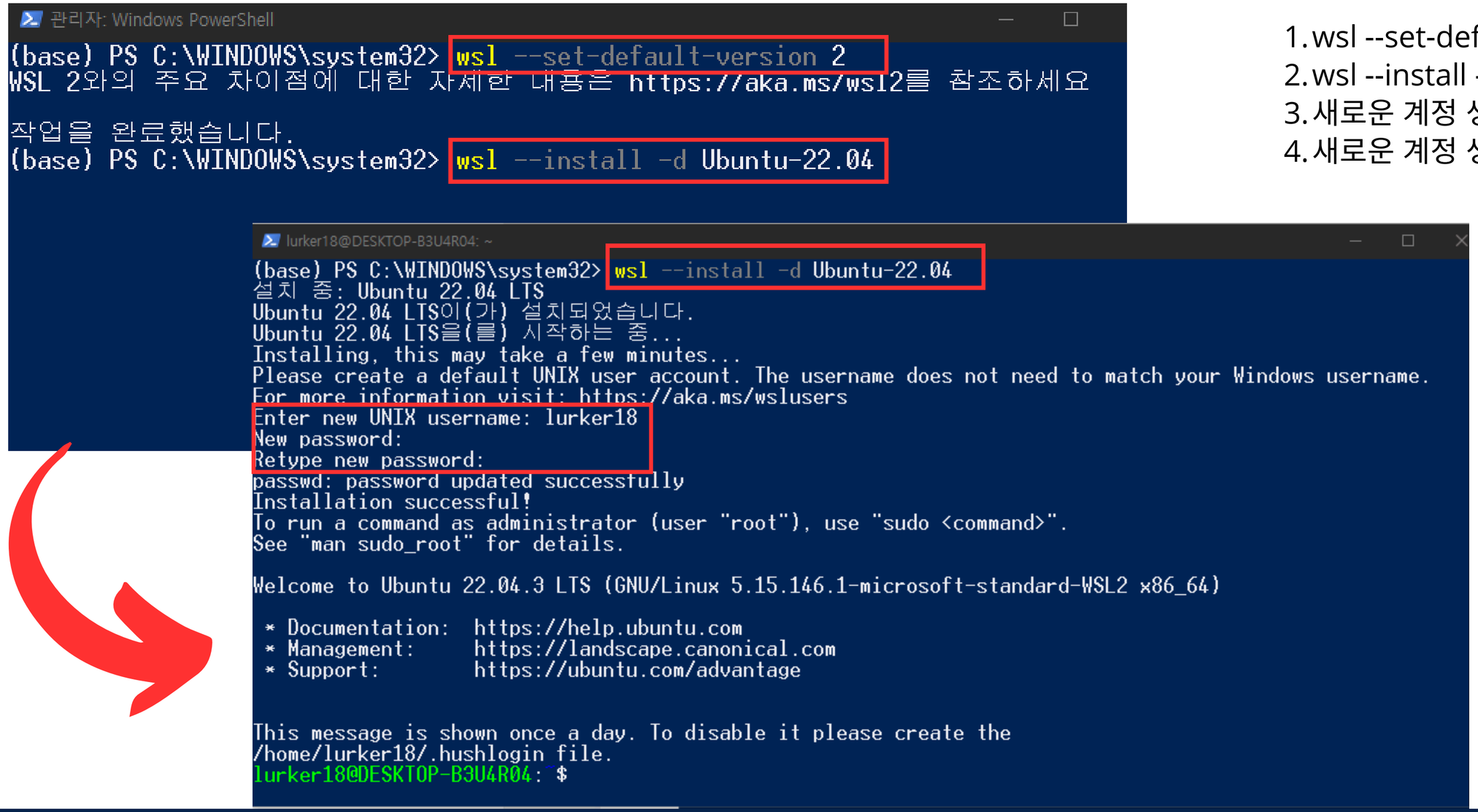
1. 윈도우 키 클릭
2. “powershell“ 입력 및 관리자로 실행 클릭



컴퓨터 다시 시작

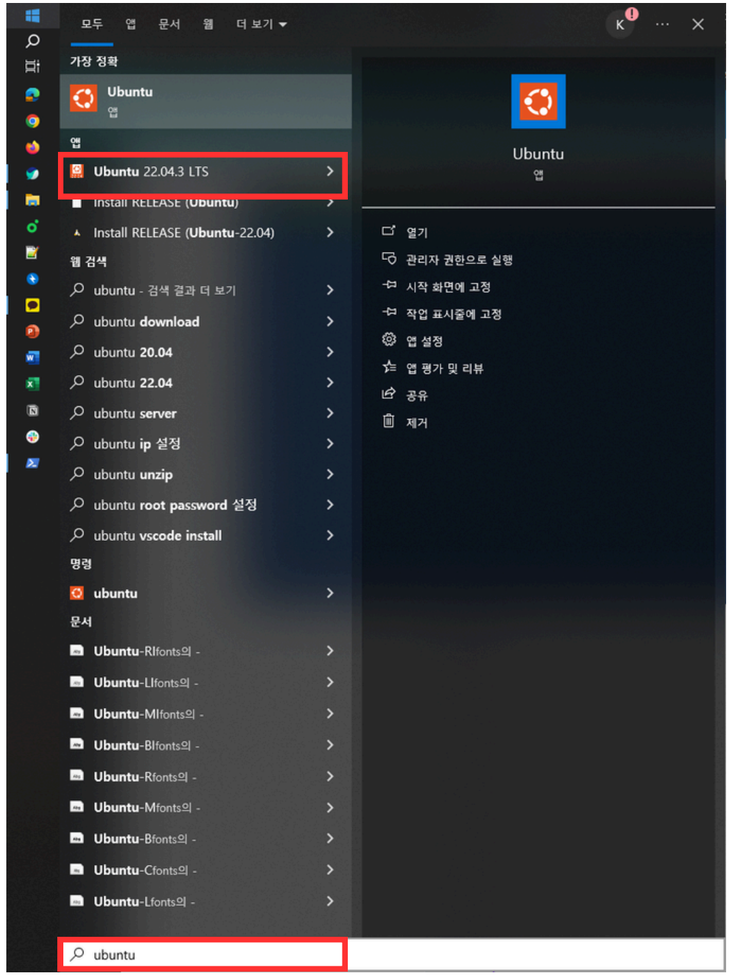
3. dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart 입력
4. dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart 입력
5. 컴퓨터 다시 시작

2. WSL2 설치 방법

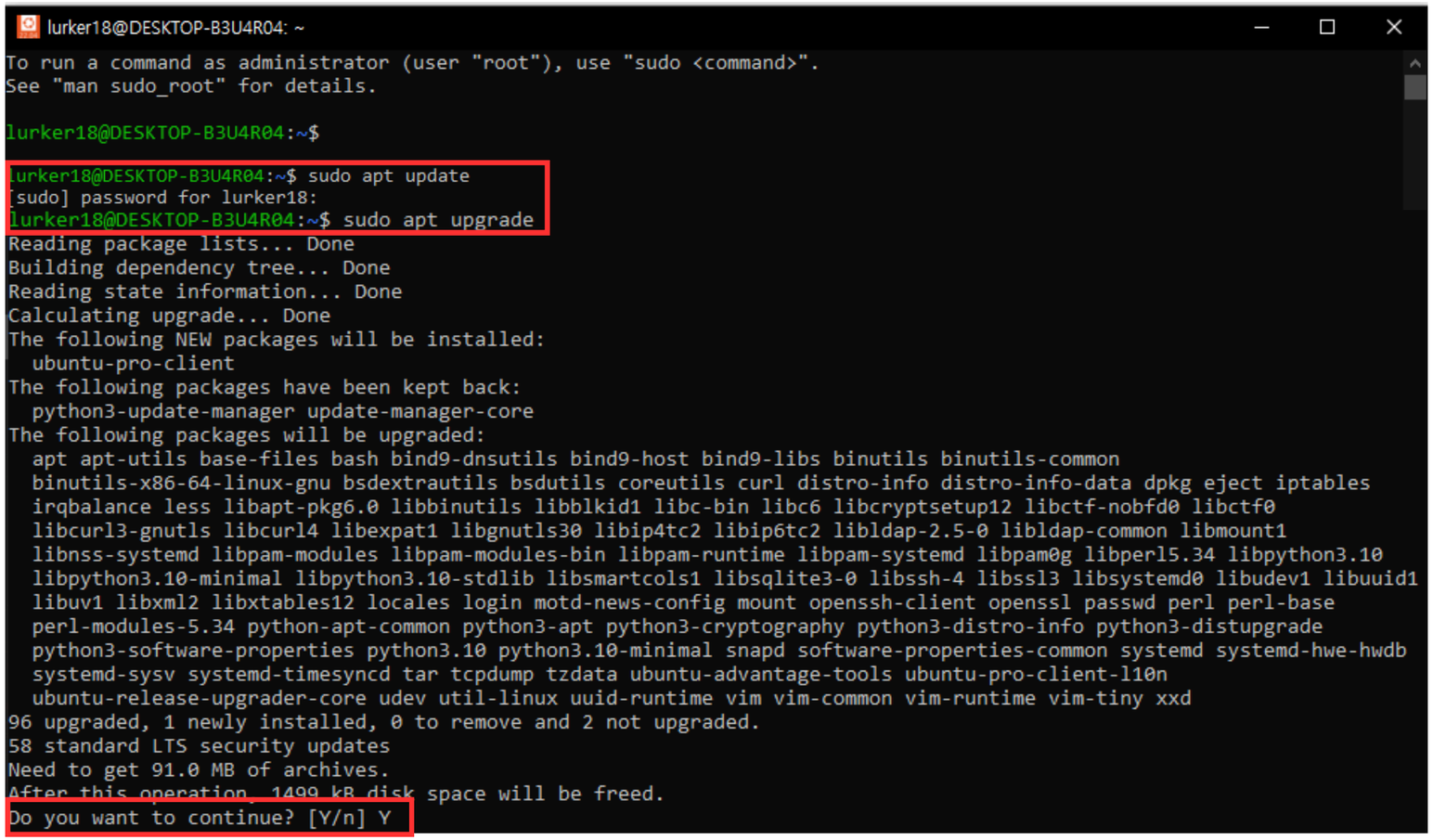


1. wsl --set-default-version 2 입력
2. wsl --install -d Ubuntu-22.04 입력
3. 새로운 계정 생성 - 사용자 ID 입력
4. 새로운 계정 생성 - 비밀번호 입력

3. 리눅스(버전: Ubuntu 22.04) 실행



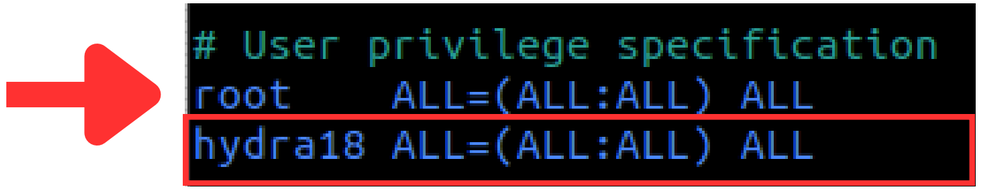
1. 윈도우 키 클릭
2. “ubuntu“ 입력 및 클릭



3. 우분투(서버 버전) 시작
4. sudo apt update 입력 (비밀번호 필요)
5. sudo apt upgrade -y 입력 (비밀번호 필요)

4. 리눅스 새로운 계정 생성 방법

```
lurker18@DESKTOP-B3U4R04:~$ ssh andlab@165.132.193.69
(base) andlab@andlab-server:~$ sudo adduser hydra18
[sudo] password for andlab: andlabyonsei
Adding user `hydra18' ...
Adding new group `hydra18' (1001) ...
Adding new user `hydra18' (1001) with group `hydra18' .
Creating home directory `/home/hydra18' ...
Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for hydra18
```



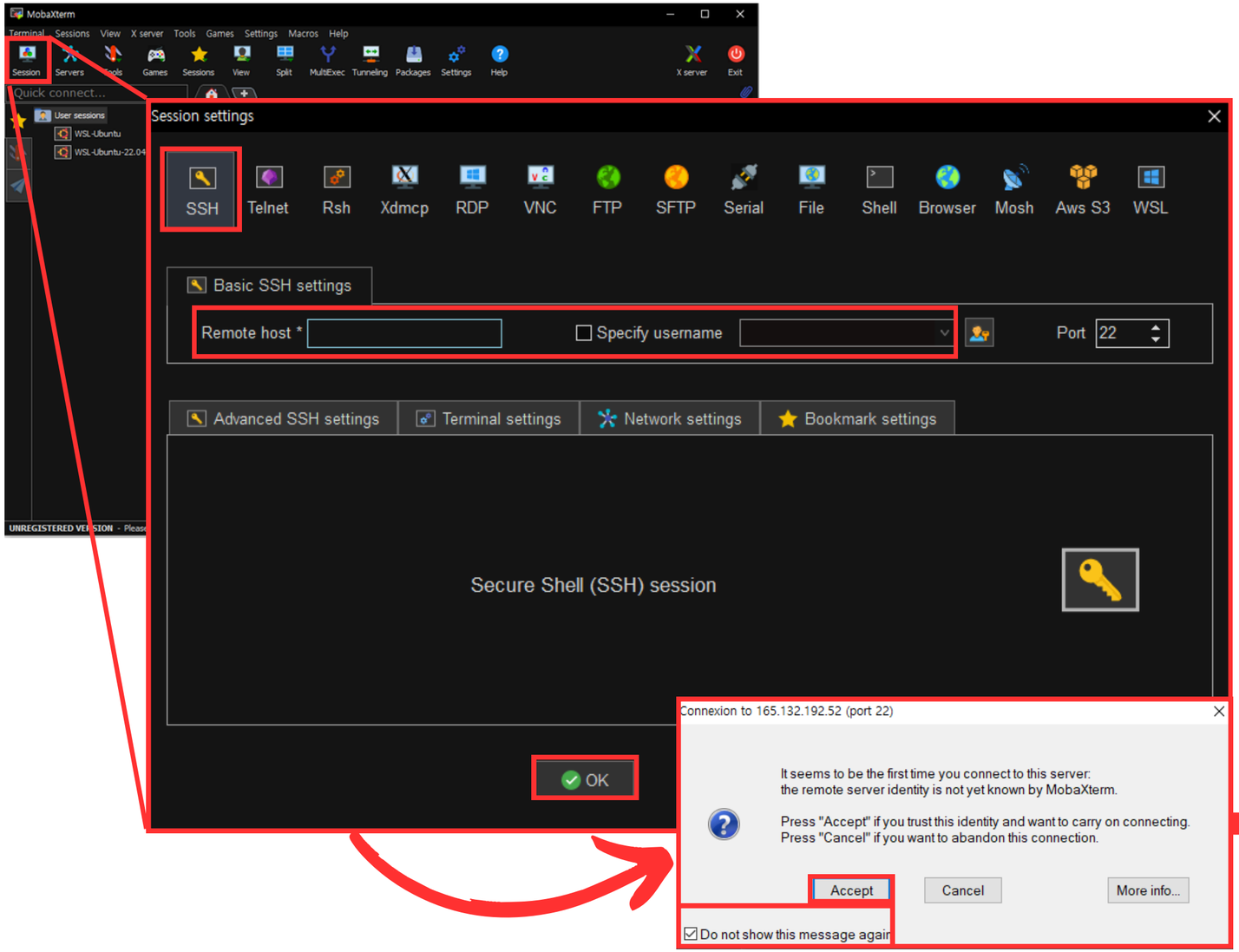
1. ssh andlab@165.132.193.69 입력 (‘andlabyonsei’라고 입력)

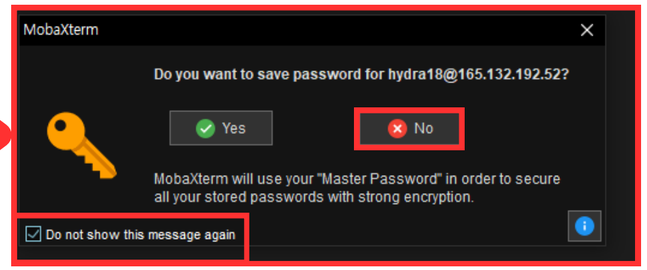
2. sudo adduser <계정명>
   andlab 계정의 비밀번호 입력 (‘andlabyonsei’라고 입력)

3. 계정을 위한 새로운 비밀번호 입력

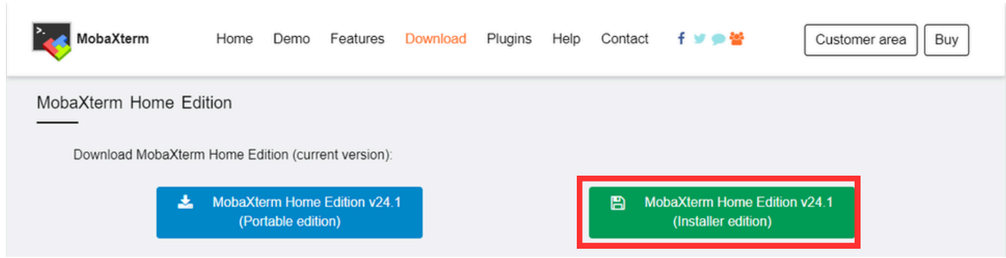
4. 새로 생성된 계정에 권한을 할당 받을것
(*관리자에게 문의 필요)

5.1 (옵션) 우분투 서버 접속 프로그램 설치 (Win 10/11 전용)





1. mobaxterm 프로그램 설치 (아래 URL접속)
https://mobaxterm.mobatek.net/download-home-edition.html



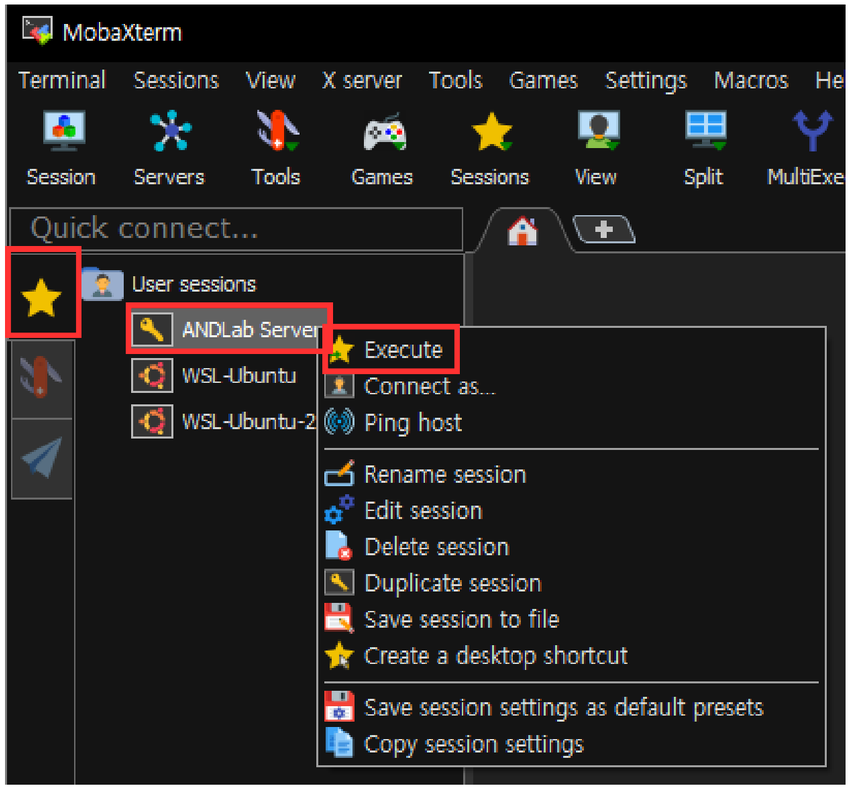
2. 다운받고 압축풀고 설치하고 프로그램 실행

3. Session 클릭 --> SSH 클릭

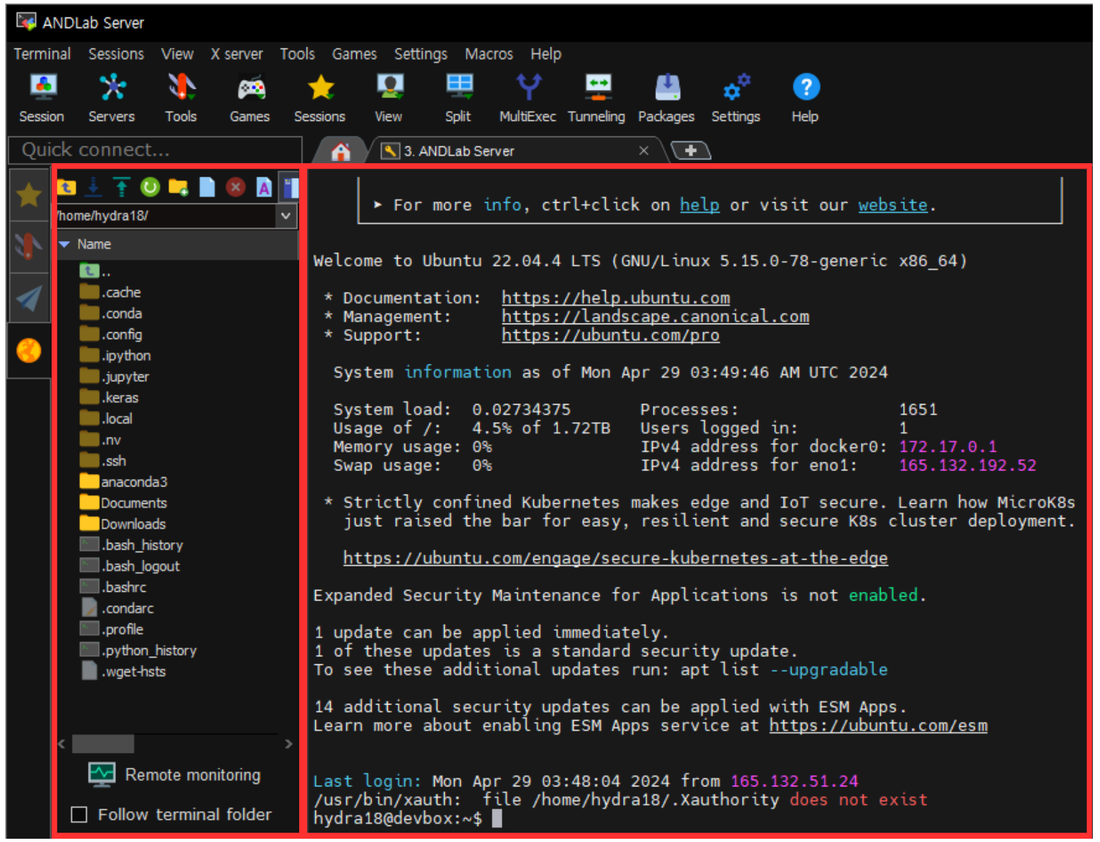
4. Remote Host: 165.132.193.69 입력
   Specify Username: <계정명> 입력

5. Accept 클릭
6. 계정 비밀번호 저장 하고 싶으면 'Yes', 아니면 'No' 클릭

5.1 (옵션) 우분투 서버 접속 프로그램 설치 (Win 10/11 전용)



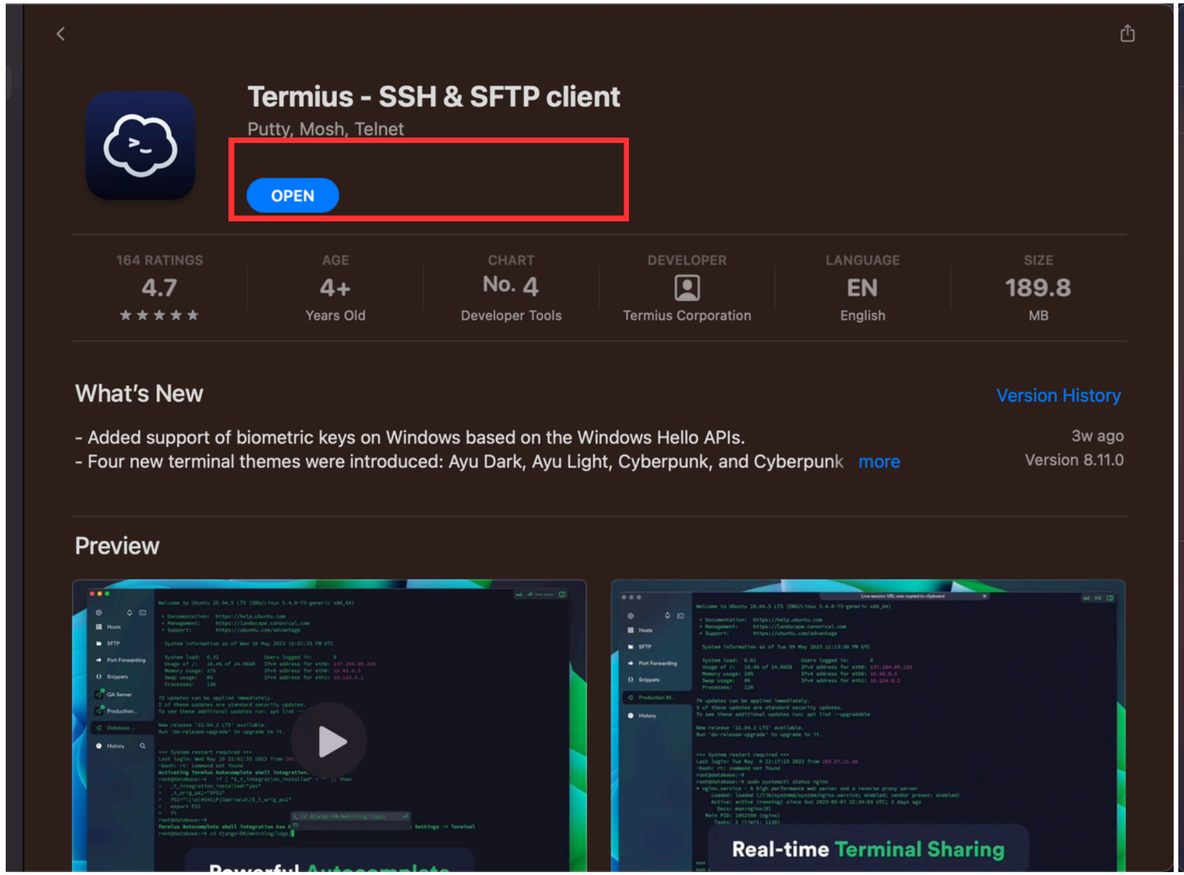
1. 별 (★) 클릭
2. 방금 만든 session명 오른쪽 마우스 클릭
3. Execute 실행 (계정명 비밀번호 입력)

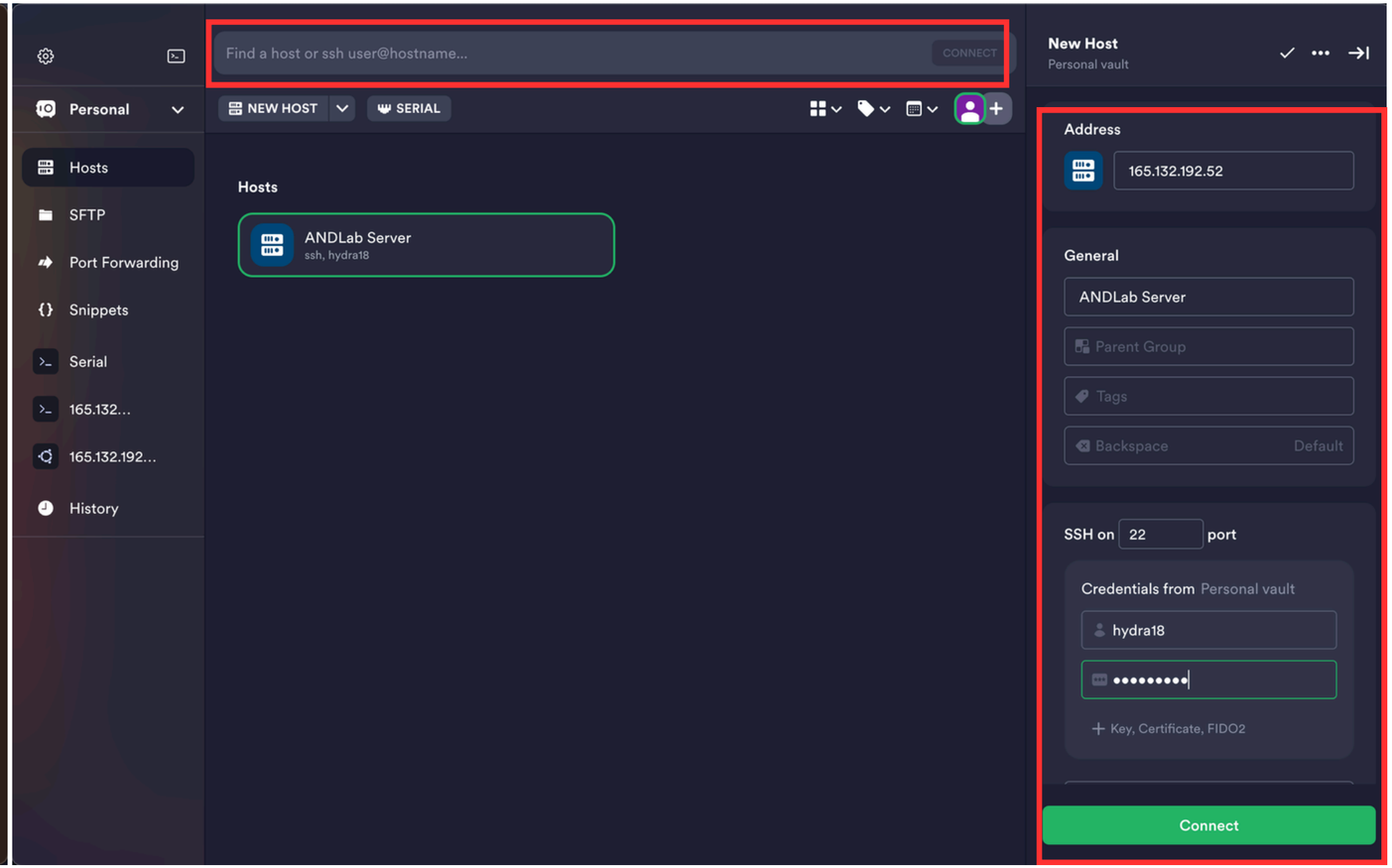


MobaXterm의 주요 기능

1. 왼쪽은 폴더 관리 (옮김, 복사/붙이기, 삭제, 폴더 생성 등 가능)
2. 오른쪽은 일반 CLI창 (서버 접속)
3. 폴더 경로 알아내기 쉽음

5.2 (옵션) 우분투 서버 접속 프로그램 설치 (Mac 전용)

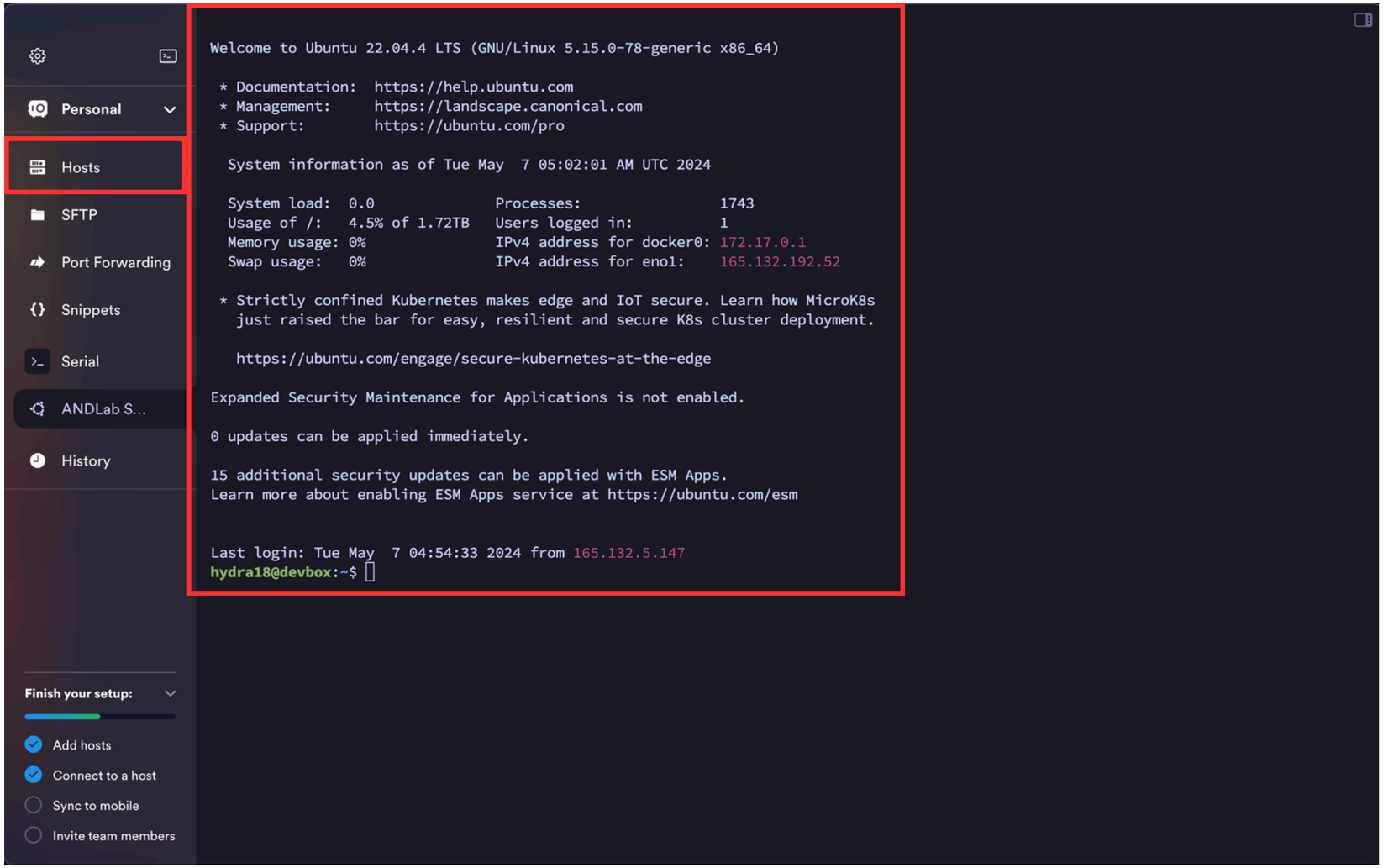




1. App Store에서 'Temius'라고 검색하고 다운받을 것
2. 간단하게 구글 계정 / 애플 계정으로 회원가입 및 로그인

3. 왼쪽 Hosts 클릭
4. 오른쪽에 서버 IP주소 (165.132.193.69) 입력
5. <계정명>이랑 비밀번호 입력

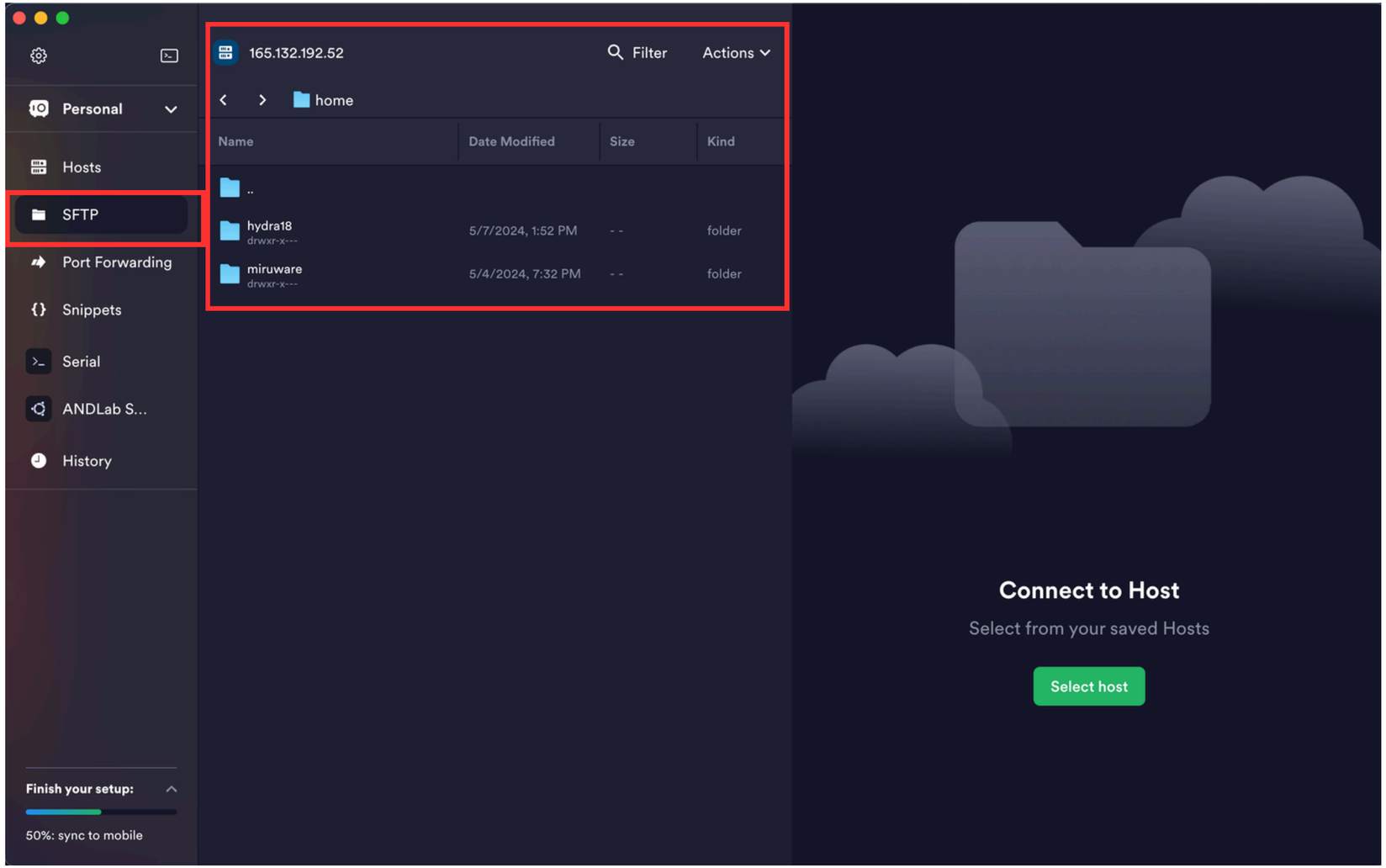
5.2 (옵션) 우분투 서버 접속 프로그램 설치 (Mac 전용)



1. 왼쪽 Hosts 클릭
2. 자동으로 서버에 로그인됨

- 만약 로그인 안될시
  ssh <계정명>@165.132.193.69 입력

5.2 (옵션) 우분투 서버 접속 프로그램 설치 (Mac 전용)



1. 왼쪽에 SFTP 클릭
2. 시각적으로 파일 관리 가능
   (폴더.파일 생성/삭제 등)

참고: 리눅스 계정 비교

> ssh andlab@165.132.193.69 | 비밀번호: andlabyonsei

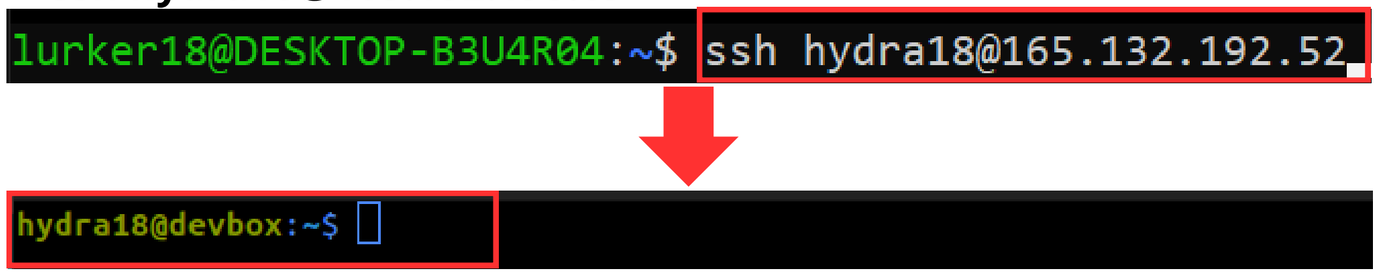


A) ssh andlab@165.132.193.69 입력
비밀번호: andlabyonsei 입력

andlab 계정: main 관리자(Administrator)
기본: Anaconda, Docker, TensorFlow, Torch, Cuda 등 설치되어 있음

VS

> ssh hydra18@165.132.193.69



B) ssh hydra18@165.132.193.69 입력
(또는 <계정명>@165.132.193.69 입력)

<계정명> 계정: 생성된 개인별 사용자 (User)
별도: Anaconda, Docker, TensorFlow, Torch 등 별도로 설치가 필요함

6.1 (andlab 계정 기준) Conda 가상환경 생성 방법

> Python 설치

```
(base)andlab@andlab-server:$ conda create -n nova18 python==3.11.8
Collecting package metadata (current_repodata.json): done
Solving environment: unsuccessful attempt using repodata from current_repodata.json, retrying with next repodata source.
Collecting package metadata (repodata.json): done
Solving environment: done


==> WARNING: A newer version of conda exists. <==
  current version: 23.5.2
  latest version: 24.3.0

Please update conda by running

    $ conda update -n base -c defaults conda

Or to minimize the number of packages updated during conda update use

     conda install conda=24.3.0


## Package Plan ##

  environment location: /home/miruware/anaconda3/envs/nova18

  added / updated specs:
    - python==3.11.8
```

1. conda create -n <가상환경명> python==<버전 번호>
 예: `conda create -n TEST python==3.11.8`

2. 설치 확인

3. conda activate <가상환경명>
 예: `conda activate TEST`
4. pip install tensorflow==2.14.0

5. pip install torch==2.1.2 torchvision==0.16.2 torchaudio==2.1.2 --index-url https://download.pytorch.org/whl/cu118

> TensorFlow 설치

```
(base) andlab@andlab-server:~$ conda activate nova18
(nova18) andlab@andlab-server:~$ pip install tensorflow==2.14.0
```

> PyTorch 설치

```
(nova18)andlab@andlab-server:~$ pip install torch==2.1.2 torchvision==0.16.2 torchaudio==2.1.2 --index-url https://download.pytorch.org
/whl/cu118
```

6.2 (andlab 계정 기준) Conda 패키지 설치 방법

> Pandas / Scikit-learn / Matplotlib / Seaborn / Scipy 설치

```
(nova18)andlab@andlab-server:~$ pip install pandas scikit-learn matplotlib seaborn scipy
```

1. pip install pandas scikit-learn matplotlib seaborn scipy

> Jupyter Notebook / Spyder 설치

```
(nova18)andlab@andlab-server:~$ conda install notebook spyder
```

2. conda install notebook spyder

```
(nova18)andlab@andlab-server:~$ mkdir nova18 && cd nova18
```

3. mkdir <계정명> && cd <계정명>

```
(nova18)andlab@andlab-server:~$ mkdir Desktop && cd Desktop
```

4. mkdir <폴더명> && cd <폴더명>

7.1 (개인 계정 기준) CUDA / CUDNN 설정

```
hydra18@devbox:~$ sudo nano ~/.bashrc
```

1. sudo nano ~/.bashrc 입력

```
#CUDA PATH
export PATH=$PATH:/usr/local/cuda/bin
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/cuda/lib:/usr/local/cuda/lib64
```

2. 가장 밑에서 아래 명령 입력하고 저장

```
# CUDA PATH
export PATH=$PATH:/usr/local/cuda/bin
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/cuda/lib:/usr/local/cuda/lib64
```

```
^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^
```

```
Save modified buffer?
 Y Yes
 N No          ^C Cancel
```

```
File Name to Write: /home/hydra18/.bashrc
^G Help                        M-D DOS Format
^C Cancel                      M-M Mac Format
```

3. 저장하는 방법

- ‘CTL+x’ 누르고
- ‘y’ 키 누르고
- ENTER 키 누른다

7.1 (개인 계정 기준) CUDA / CUDNN 설정

```
hydra18@devbox:~$ source ~/.bashrc
```

4. 저장한 소스 적용 방법
   - source ~/.bashrc 입력

```
hydra18@devbox:~$ nvcc -V
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2022 NVIDIA Corporation
Built on Wed_Sep_21_10:33:58_PDT_2022
Cuda compilation tools, release 11.8, V11.8.89
Build cuda_11.8.r11.8/compiler.31833905_0
```

5. CUDA 확인
   - nvcc -V

```
hydra18@devbox:~/Downloads$ cat /usr/local/cuda/include/cudnn_version.h | grep CUDNN_MAJOR -A 2
#define CUDNN_MAJOR 8
#define CUDNN_MINOR 9
#define CUDNN_PATCHLEVEL 7
--
#define CUDNN_VERSION (CUDNN_MAJOR * 1000 + CUDNN_MINOR * 100 + CUDNN_PATCHLEVEL)
```

6. CUDNN 확인

- cat /usr/local/cuda/include/cudnn_version.h | grep CUDNN_MAJOR -A 2 입력하고 확인
- 끝 숫자가 '8','9','7' 이라고 되어 있으면 CUDNN 버전 8.9.7 이란 뜻

7.2 (개인 계정 기준) Anaconda 설치

```
hydra18@devbox:~$ mkdir Downloads && cd Downloads
hydra18@devbox:~/Downloads$
```

1. 새로운 폴더 생성함
   mkdir Downloads && cd Downloads 입력

```
hydra18@devbox:~/Downloads$ sudo apt-get update
[sudo] password for hydra18:
Get:1 https://download.docker.com/linux/ubuntu jammy InRelease [48.8 kB]
Hit:2 https://nvidia.github.io/libnvidia-container/stable/deb/amd64  InRelease
Get:3 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64
Get:4 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64
Hit:5 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [119 kB]
```

2. sudo apt-get update 입력

```
hydra18@devbox:~/Downloads$ wget https://repo.anaconda.com/archive/Anaconda3-2024.02-1-Linux-x86_64.sh
```

3. wget https://repo.anaconda.com/archive/Anaconda3-2024.02-1-Linux-x86_64.sh 입력

```
hydra18@devbox:~/Downloads$ bash Anaconda3-2024.02-1-Linux-x86_64.sh
```

4. bash Anaconda3-2024.02-1-Linux-x86_64.sh

```
Welcome to Anaconda3 2024.02-1

In order to continue the installation process, please review the license
agreement.
Please, press ENTER to continue
>>>
```

5. 'ENTER' 키 계속 누르시고 진행하면됨

```
Anaconda3 will now be installed into this location:
/home/hydra18/anaconda3

  - Press ENTER to confirm the location
  - Press CTRL-C to abort the installation
  - Or specify a different location below

[/home/hydra18/anaconda3] >>>
PREFIX=/home/hydra18/anaconda3
Unpacking payload ...
Extracting : distro-1.8.0-py311h06a4308_0.conda:  53%|
```

7.2 (개인 계정 기준) Anaconda 설치

> 가상환경 생성 및 Python 설치

```
hydra18@devbox:~$ conda create -n tensor python=3.11.8
```

```
hydra18@devbox:~$ conda activate tensor
```

1. conda create -n <가상환경명> python=<버전번호>
2. conda activate <가상환경명>

> TensorFlow 설치

```
(tensor) hydra18@devbox:~$ pip install tensorflow==2.14.0
```

3. pip install tensorflow==2.14.0

> PyTorch 설치

```
(tensor) hydra18@devbox:~$ pip install torch==2.1.2 torchvision==0.16.2 torchaudio==2.1.2 --index-url https://download.pytorch.org/whl/cu118
```

4. pip install torch==2.1.2 torchvision==0.16.2 torchaudio==2.1.2 --index-url https://download.pytorch.org/whl/cu118

> Pandas / Scikit-learn / Matplotlib / Seaborn / Scipy 설치

```
(tensor) hydra18@devbox:~$ pip install pandas scikit-learn matplotlib seaborn scipy
```

5. pip install pandas scikit-learn matplotlib seaborn scipy

> Jupyter Notebook / Spyder 설치

```
(tensor) hydra18@devbox:~$ conda install notebook spyder
```

6. conda install notebook spyder

```
(tensor) hydra18@devbox:~$ mkdir tensor && cd tensor
```

7. mkdir <가상환경명> && cd <가상환경명>

```
(tensor) hydra18@devbox:~$ mkdir temp && cd temp
```

8. mkdir <폴더명> && cd <폴더명>

참고: Anaconda 환경 비교

```
base)andlab@andlab-server:~$
```

```
(base)andlab@andlab-server:~$ python --version
Python 3.10.14
```

```
>>> tf.__version__
'2.14.0'
```

```
>>> torch.__version__
'2.1.2'
```

andlab 계정: main 관리자(Administrator)
기본: Anaconda, Docker, TensorFlow, Torch, Cuda 등 설치되어 있음

Python ver.: 3.10.14

TensorFlow ver.: 2.14.0

Pytorch ver.: 2.1.2

VS

```
(base) hydra18@devbox:~$
```

```
(tensor) hydra18@devbox:~$ python --version
Python 3.11.8
```

```
>>> tf.__version__
'2.14.0'
```

```
>>> torch.__version__
'2.1.2'
```

<계정명> 계정: 계정 사용자(User)
설치함: Anaconda, TensorFlow, Torch, NumPy 등 설치하면 됨

- 다음 페이지 참조

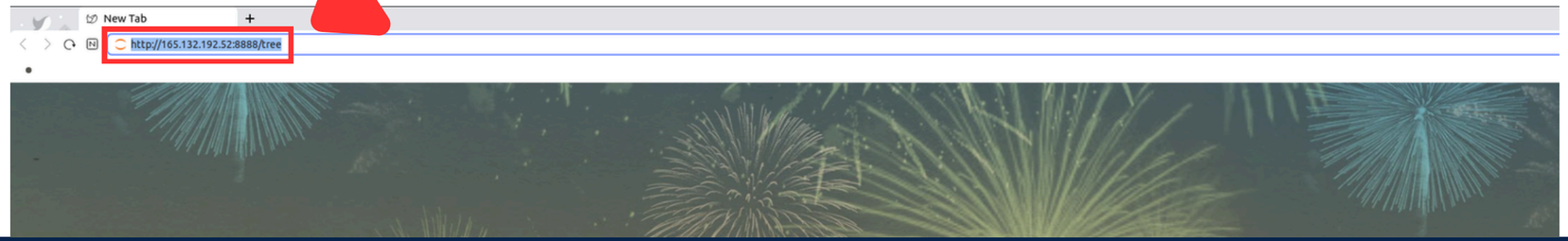
Python ver.: 3.11.8

TensorFlow ver.: 2.14.0

Pytorch ver.: 2.1.2

8. (모든 계정 가능) Jupyter-Notebook 설치 및 실행 방법

```
(nova18) miruware@devbox:~/nova18/Desktop/code$ jupyter-notebook
[I 2024-04-23 09:17:37.644 ServerApp] Package notebook took 0.0000s to import
[I 2024-04-23 09:17:37.649 ServerApp] Package jupyter_lsp took 0.0042s to import
[W 2024-04-23 09:17:37.649 ServerApp] A `_jupyter_server_extension_points` function was not found in jupyter_lsp. Instead, a
ter_server_extension_paths` function was found and will be used for now. This function name will be deprecated in future rel
f Jupyter Server.
[I 2024-04-23 09:17:37.651 ServerApp] Package jupyter_server_terminals took 0.0019s to import
[I 2024-04-23 09:17:37.651 ServerApp] Package jupyterlab took 0.0000s to import
[I 2024-04-23 09:17:37.664 ServerApp] Package notebook_shim took 0.0000s to import
[W 2024-04-23 09:17:37.664 ServerApp] A `_jupyter_server_extension_points` function was not found in notebook_shim. Instead,
pyter_server_extension_paths` function was found and will be used for now. This function name will be deprecated in future r
 of Jupyter Server.
[I 2024-04-23 09:17:37.664 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2024-04-23 09:17:37.666 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2024-04-23 09:17:37.668 ServerApp] jupyterlab | extension was successfully linked.
[W 2024-04-23 09:17:37.670 ServerApp] ServerApp.password config is deprecated in 2.0. Use PasswordIdentityProvider.hashed_pa
[I 2024-04-23 09:17:37.670 ServerApp] notebook | extension was successfully linked.
[I 2024-04-23 09:17:37.781 ServerApp] notebook_shim | extension was successfully linked.
[I 2024-04-23 09:17:37.790 ServerApp] notebook_shim | extension was successfully loaded.
[I 2024-04-23 09:17:37.791 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2024-04-23 09:17:37.791 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[I 2024-04-23 09:17:37.792 LabApp] JupyterLab extension loaded from /home/miruware/anaconda3/envs/nova18/lib/python3.11/site
es/jupyterlab
[I 2024-04-23 09:17:37.792 LabApp] JupyterLab application directory is /home/miruware/anaconda3/envs/nova18/share/jupyter/la
[I 2024-04-23 09:17:37.792 LabApp] Extension Manager is 'pypi'.
[I 2024-04-23 09:17:37.793 ServerApp] jupyterlab | extension was successfully loaded.
[I 2024-04-23 09:17:37.794 ServerApp] notebook | extension was successfully loaded.
[I 2024-04-23 09:17:37.794 ServerApp] Serving notebooks from local directory: /home/miruware/nova18/Desktop/code
[I 2024-04-23 09:17:37.794 ServerApp] Jupyter Server 2.10.0 is running at:
[I 2024-04-23 09:17:37.794 ServerApp] http://165.132.192.52:8888/tree
[I 2024-04-23 09:17:37.794 Server      http://127.0.0.1:8888/tree
[I 2024-04-23 09:17:37.79  App] Use Control-C to stop this server and shut down all kernels (twice to skip confirmatio
```



1. jupyter-notebook 입력

2. 아래 IP주소 복사하고 host쪽의 브라우저에 붙이기
예: 165.132.193.69:8888 입력

3. 비밀번호 조건:
    if andlab계정:
        andlabyonsei 입력
    else (다른 계정들):
        (빈값 입력) 또는 (계정별 비밀번호 입력)

FAQ: (terminal에서) 종료 필요시: ctl + c --> y

9. [Version 1](모든 계정 가능) VS-Code 실행 방법

```
hydra18@devbox:~$ dpkg -l | grep code-server
ii  code-server                          4.23.1
   amd64        Run VS Code in the browser.
```

```
hydra18@devbox:~$ code-server
```

```
hydra18@devbox:~$ nano ~/.config/code-server/config.yaml
```

```
  GNU nano 6.2                    /home/hydra18/.config/code-server/config.yaml
bind-addr: 0.0.0.0:9090
auth: password
password:
cert: false
```

1. dpkg -l | grep code-server
입력하고 설치가 정상적으로 설치됐는지 확인함

2. code-server 입력하고 실행 한번 한다.

3. nano ~/.config/code-server/config.yaml 입력

4. bind-addr: 0.0.0.0:9090 (기본 default port번호: 8080)
으로 수정

주의: port번호는 각 계정 마다 새로운 포터 번호 정의 해야함)
즉, 8080/9090 외 개인 포트 번호 필요!

password: (원하는 비밀번호 변경)

9. [Version 1](모든 계정 가능) VS-Code 실행 방법

```
hydra18@devbox:~$ nano ~/.config/code-server/config.yaml
hydra18@devbox:~$ sudo systemctl enable --now code-server@hydra18
```

5. sudo systemctl enable --now code-server@<계정명> 입력

```
hydra18@devbox:~$ sudo systemctl list-units code-server*
  UNIT                          LOAD   ACTIVE SUB     DESCRIPTION
  code-server@hydra18.service loaded active running code-server
```

6. sudo systemctl list-units code-server* 입력

```
hydra18@devbox:~$ sudo systemctl stop code-server@hydra18.service
```

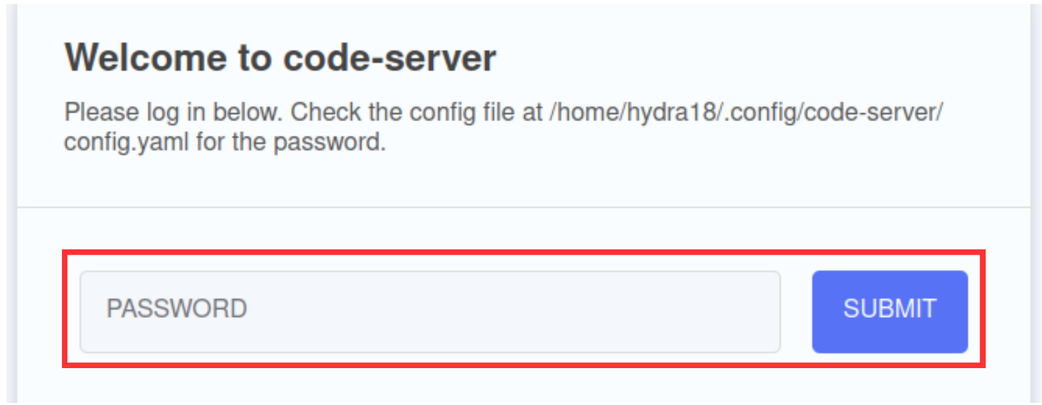
- 만약 VS-Code server 중지가 필요하면 옆 명령어 입력하면됨
- sudo systemctl stop code-server@<계정명>.service

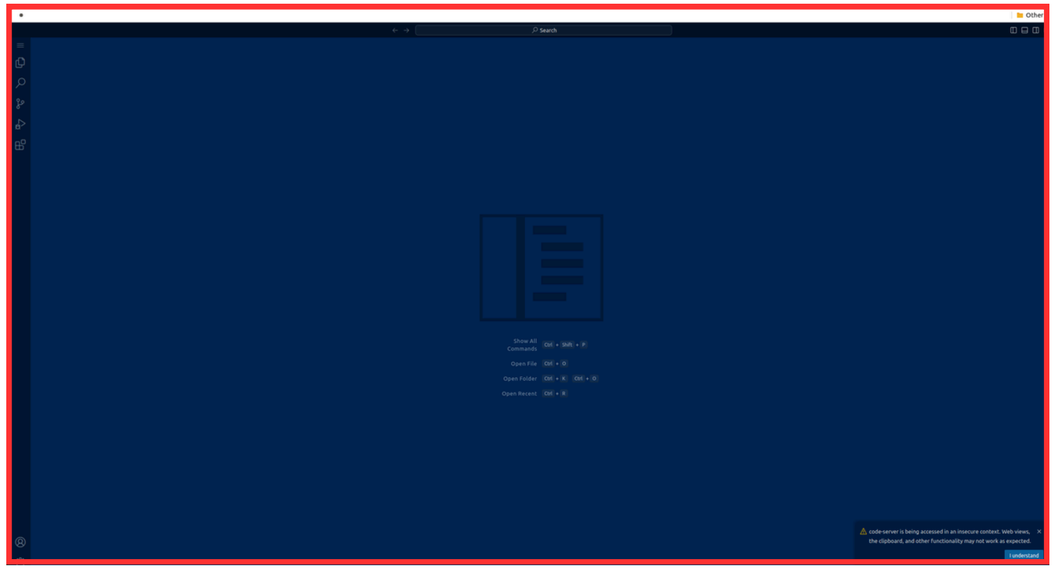
```
hydra18@devbox:~$ sudo systemctl disable code-server@hydra18
```

- 비활성화 (시스템 부팅 시 자동으로 시작 X)
- sudo systemctl disable code-server@<계정명>

9. [Version 1](모든 계정 가능) VS-Code 실행 방법





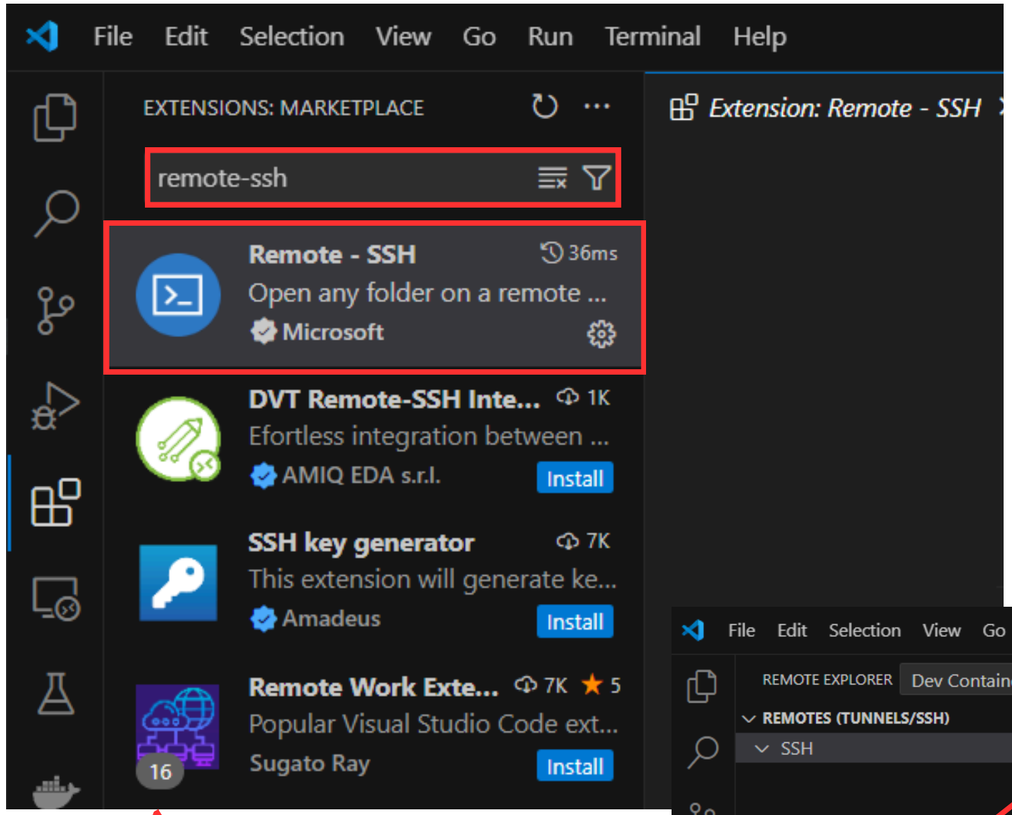


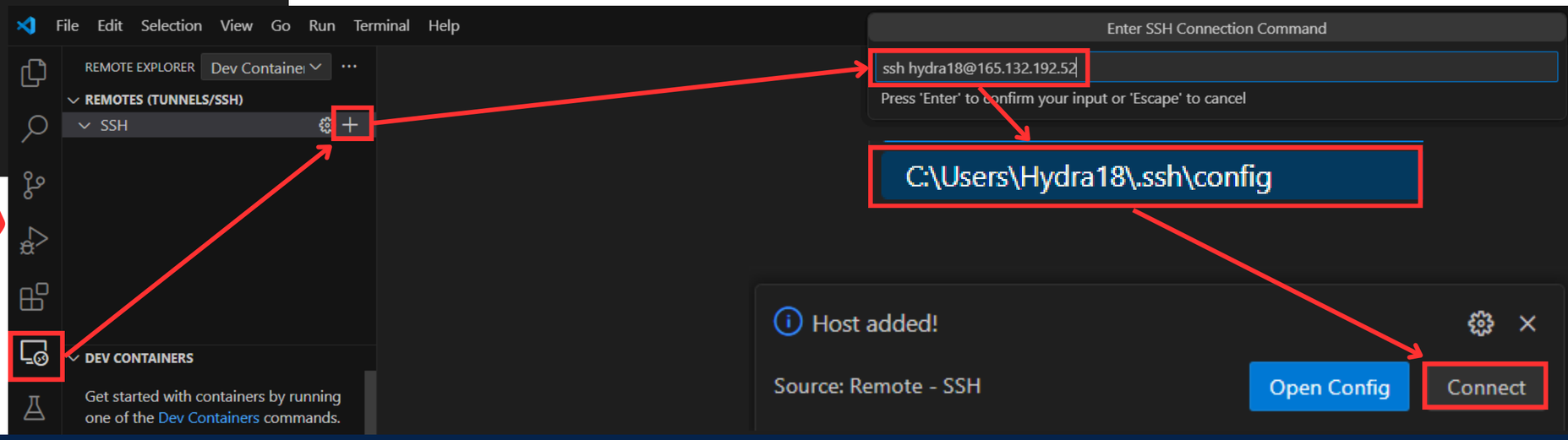
1. 165.132.193.69:포트번호 브라우저에 입력
예: Port 번호: 8080, 9090 등 (계정 마다 다름)

2. 비밀번호 입력
참조: (.config/code-server/config.yaml에서 설정한 비밀번호 입력)

3. VS-Code(Server)가 실행 되며 코드 작성하면 됨

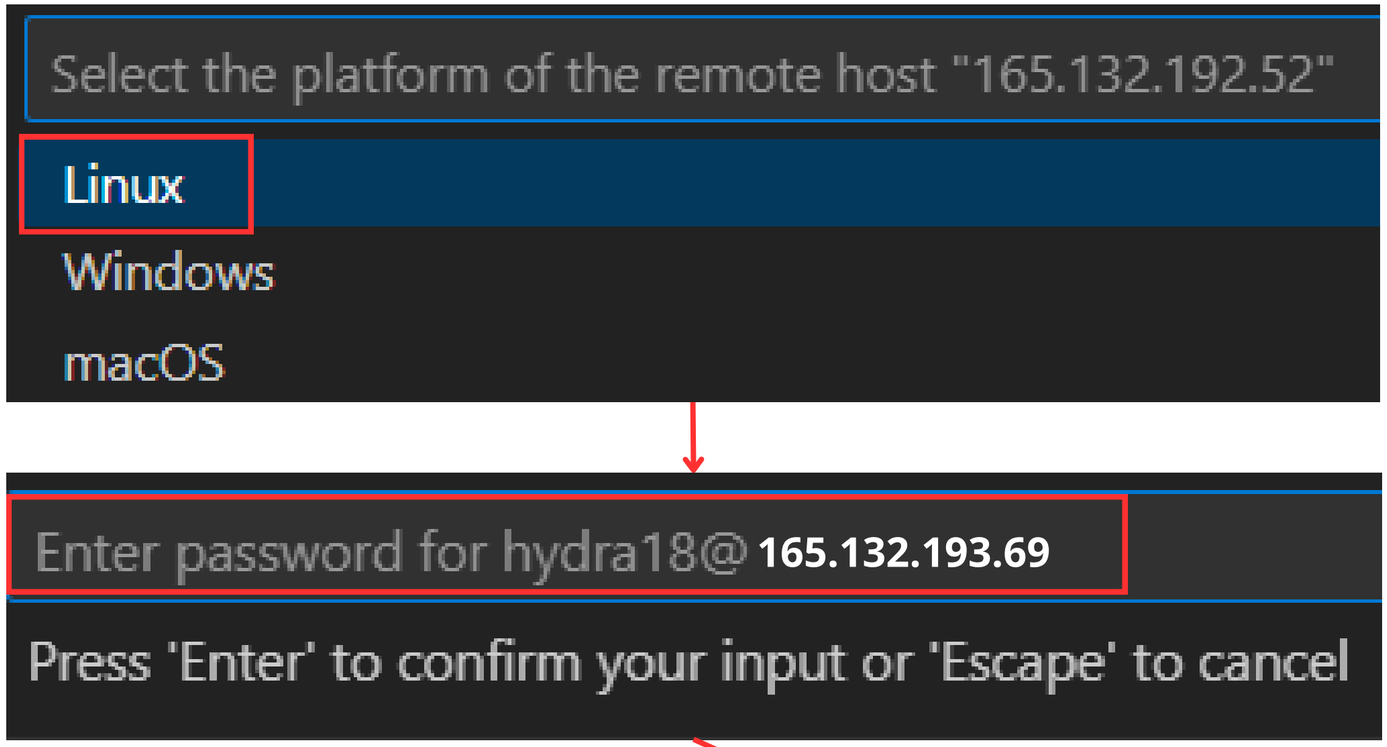
9. [Version 2](개인 계정 기준) VS-Code 실행 방법



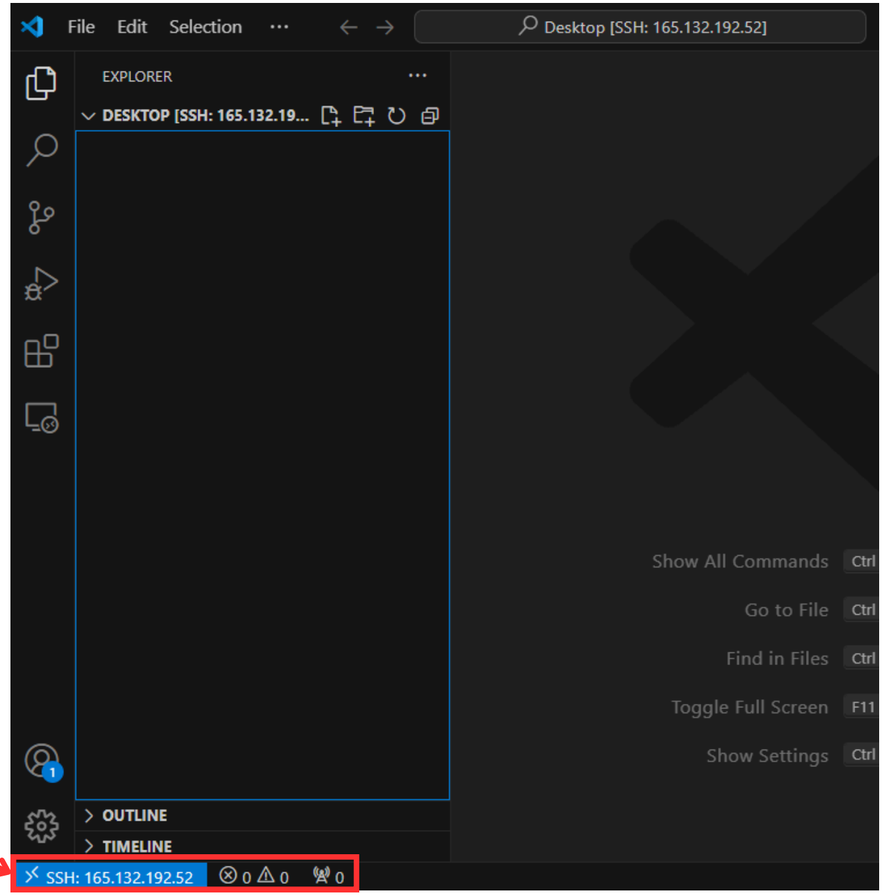


1. (Host기준) “Remote-SSH” 라고 검색
2. Remote-SSH 설치함.
3. 왼쪽 아래 ‘remote-ssh’ 아이콘 클릭
4. REMOTES(TUNNELS/SSH) 부분에 SSH옆에 ‘+’ 클릭
5. ssh <계정명>@165.132.193.69 입력 후 enter
6. C:\Users\<계정명>\.ssh\config 클릭
7. 아래 Connect 클릭

9. [Version 2](개인 계정 기준) VS-Code 실행 방법



8. Linux 선택
9. (원격 기준) 원격 계정 비밀번호 입력
10. 아래 SSH: 165.132.193.69가 뜨면 연결 완료



10. 전체 디스크 확인 방법

1. sudo lsblk -o NAME,FSTYPE,SIZE,MOUNTPOINT,LABEL 입력하고 전체 디스크 확인

```
sudo lsblk -o NAME,FSTYPE,SIZE,MOUNTPOINT,LABEL
[sudo] password for miruware:
NAME        FSTYPE   SIZE MOUNTPOIN LABEL
nvme0n1                14T
├─nvme0n1p1 vfat      512M /boot/efi
└─nvme0n1p2 ext4       14T /
nvme1n1              894.3G
```

기본 드라이브: 총 14TB = /home/(계정명)/

추가 드라이브: 총 1TB = /mnt/nvme01 (총 1TB)



- **데이터 저장은 왠만하면 nvme01 폴더 사용할것**

  경로: /mnt/nvme01

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> AllenAI 폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
AllenAI/longformer-base-4096	약 149M	Float 32-bit	2개 이상 필요	> 24GB
AllenAI/longformer-large-4096	약 149M	Float 32-bit	2개 이상 필요	> 24GB

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> DistilBERT 폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
DistilBERT/distilbert-base-cased	65.8M	Float 32-bit	1개 필요	> 8GB
DistilBERT/distilbert-base-uncased	65.8M	Float 32-bit	1개 필요	> 8GB
DistilBERT/distilroberta-base	82.8M	Float 32-bit	1개 필요	> 8GB
DistilBERT/distilgpt2	88.2M	Float 32-bit	1개 필요	> 8GB
DistilBERT/distilbert-base-multilingual-cased	135M	Float 32-bit	1개 필요	> 12GB

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> Dmis-Lab 폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
Dmis-Lab/biobert-v1.1	110M	Float 32-bit	1개 필요	> 8GB

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> Epfl-llm폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
Epfl-llm/meditron-7b	6.74B	BFloat 16-bit	1개 필요	> 24GB
Epfl-llm/meditron-70b	69B	BFloat 16-bit	8개 이상 필요	>=384GB

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> Facebook폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
Facebook/bart (base / large)	139M / 약 406M	Float 32-bit	1개 필요	> 11GB
Facebook/dino (base / large / giant)	86.6M / 304M / 1.14B	Float 32-bit	1개 필요	> 16GB
Facebook/dpr (CTX / Question)	N/A	Float 32-bit	1개 필요	> 11GB
Facebook/opt-125m	125M	N/A	1개 필요	> 11GB
Facebook/opt-350m	350M	N/A	1개 필요	> 11GB
Facebook/opt-1.3b	1.3B	N/A	1개 필요	> 16GB
Facebook/opt-2.7b	2.7B	N/A	1개 필요	> 16GB

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> FacebookAI폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
FacebookAI/roberta-base	125M	Float 32-bit	1개 필요	>11GB
FacebookAI/roberta-large	355M	Float 32-bit	1개 필요	>12GB
FacebookAI/xlm-roberta-base	279M	Float 32-bit	1개 필요	>12GB
FacebookAI/xlm-roberta-large	561M	Float 32-bit	1개 필요	>12GB

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> Google폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
Google/bert (base / large / multilingual)	110M / 336M / 179M	Float 32-bit	1개 필요	>8GB
Google/codegemma (2b / 7b)	2.51B / 8.54B	BFloat 16-bit	2개 이상 필요	>=24GB
Google/electra-base	N/A	Float 32-bit	1개 필요	>11GB
Google/flan-t5 (small / base / large /xl / xxl)	77M ~ 11.3B	Float 32-bit	1개 ~ 3개 이상 필요	>=8GB
Google/gemma (2b / 7b)	2.51B ~ 8.54B	BFloat 16-bit	2개 이상 필요	>=24GB
Google/mt5 (base / large / xl / xxl)	N/A	Float 32-bit	1개 ~ 3개 이상 필요	>=8GB
Google/paligemma-3b (mix / pt)	2.92B	Float 32-bit	1개 필요	>16GB

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> Google폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
Google/siglip (base / large)	203M / 652M	Float 32-bit	1개 필요	>12GB
Google/t5 (base / large)	223M / 738M	Float 32-bit	1개 필요	>16GB
Google/t5-v1.1 (base / large / xl / xxl)	77M ~ 11.3B	Float 32-bit	1개 ~ 3개 이상 필요	>=8GB
Google/umt5 (small / base / xl)	N/A	Float 32-bit	1개 필요	>16GB
Google/vit-base-patch16	86.6M	Float 32-bit	1개 필요	>12GB

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> MetaAI폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
MetaAI/Llama_3.1 (8B / 70B) (Base / Instruct)	6.74B / 69B	BFloat 16-bit	1개 ~ 8개 이상 필요	>=24GB
MetaAI/Llama_3.2 (1B / 3B / 11B / 90B) (Base / Instruct / Vision)	1B / 3B / 10.9B / 89.7B	Float 32-bit / 16-bit	1개 ~ 8개 이상 필요	>=24GB
MetaAI/Llama_3.3 (70B) (Instruct)	70.6B	BFloat 16-bit	3개 ~ 8개 이상 필요	>=24GB

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> Microsoft폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
Microsoft/Beit (Base / Large)	87M 이상	Integer 64-bit / Float 32-bit	1개 필요	>11GB
Microsoft/biogpt	347M	N/A	1개 필요	>16GB
Microsoft/Deberta (Base / Large / XLarge)	N/A	N/A	1개 필요	>16GB
Microsoft/Deberta-v3 (Small / Base / Large)	N/A	N/A	1개 필요	>16GB
Microsoft/DialoGPT (Small / Medium / Large)	147M 이상	N/A	1개 필요	>16GB
Microsoft/Florence-2 (Base / Large)	0.23B / 0.77B	N/A	1개 필요	>16GB

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> Microsoft폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
Microsoft/GIT(Base / Large)	177M 이상	Integer 64-bit / Float 32-bit	1개 필요	>16GB
Microsoft/LayoutLM (v1 / v2 / v3)	125M 이상	Integer 64-bit / Float 32-bit	1개 필요	>11GB
Microsoft/Phi (3.5 / 4) (Mini / Medium / Small / Multimodal)	7B, 14B	BFloat 16-bit	1개 필요	>24GB
Microsoft/BiomedNLP-BiomedBERT-base	N/A	N/A	1개 필요	>11GB
Microsoft/table-transformer (Structure / Detection)	28.8M	Float 32-bit	1개 필요	>11GB

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> MistralAI폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
MistralAI/Mistral-7B (Base / Instruct) (v0.1 / v0.2 / v0.3)	7.25B	BFloat 16-bit	1개 필요	>24GB
MistralAI/Mixtral-8x7B (Base / Instruct)	46.7B	BFloat 16-bit	8개 이상 필요	>=384GB

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> OpenAI폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
OpenAI/clip-vit (Base / Large) (Patch16 / Patch32 / Patch14)	428M 이상	Integer 64-bit / Float 32-bit	1개 필요	>24GB
OpenAI/GPT1	120M	Float 32-bit	1개 필요	>11GB
OpenAI/GPT2 (Base / Medium / Large / XL)	137M ~ 1.61B	Float 32-bit	1개 필요	>24GB
OpenAI/whisper (Tiny / Base / Medium / Large)	37.8M ~ 1.54B	Float 16-bit	1개 필요	>16GB

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> Salesforce폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
Salesforce/blip2-flan-t5-xl	3.94B	Float 32-bit	1개 필요	>48GB
Salesforce/blip2-opt-2.7b	3.84B	Float 32-bit	1개 필요	>48GB
Salesforce/blip2-opt-6.7b	7.96B	Float 32-bit	2개 이상 필요	>=96GB
Salesforce/blip-image-captioning-base	N/A	Integer 64-bit / Float 32-bit	1개 필요	>24GB
Salesforce/blip-image-captioning-large	470M	Integer 64-bit / Float 32-bit	1개 필요	>24GB
Salesforce/blip-vqa-base	385M	Integer 64-bit / Float 32-bit	1개 필요	>24GB

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> Sentence-Transformers폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
Sentence-Transformers/MiniLM (L6 / L12)	22.7M ~ 33.4M	Integer 64-bit / Float 32-bit	1개 필요	> 11GB
Sentence-Transformers/mpnet-base	109M	Integer 64-bit / Float 32-bit	1개 필요	> 11GB
Sentence-Transformers/gtr-t5-xxl	4.86B	Float 16-bit	1개 필요	> 24GB
Sentence-Transformers/LaBSE	N/A	N/A	1개 필요	> 11GB
Sentence-Transformers/paraphrase-MiniLM-L6-v2	22.7M	Float 32-bit / Integer 64-bit	1개 필요	> 11GB
Sentence-Transformers/paraphrase-multilingual-MiniLM-L12-v2	118M	Integer 64-bit / Float 32-bit	1개 필요	> 11GB

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> StabilityAI폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
StabilityAI/stable-diffusion-3-medium	2B	N/A	1개 필요	>24GB
StabilityAI/stable-diffusion-xl-base-1.0	N/A	N/A	N/A	N/A

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> TinyLlama폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
TinyLlama/TinyLlama-1.1B-Chat-v1.0	1.1B	BFloat 16-bit	1개 필요	>11GB

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/models 경로

> Vaiv폴더

모델명	**# of Params**	**Tensor Type**	**# of GPU**	**# of Mem**
GeM2-Llamion-14B-Base	14.5B	BFloat 16-bit	2개 이상 필요	>=96GB
GeM2-Llamion-14B-Chat	14.5B	BFloat 16-bit	2개 이상 필요	>=96GB
llamion-14b-base	14.5B	BFloat 16-bit	2개 이상 필요	>=96GB
llamion-14b-chat	14.5B	BFloat 16-bit	2개 이상 필요	>=96GB

11. 공용 데이터 및 모델 (읽기/쓰기)

MistralAI의 MistralAI-7B 기준

예: 만약 'Mistral-7B-Instruct-v0.3' 모델을 읽고 싶으면, 다음과 같이 설정을 하면됨.

1. model_dir = "/home/shared/huggingface/models" 변수 하나 선언함
2. 위 변수에 맞춰 사용하고자 하는 모델을 지정함. (예: model_dir + '/MistralAI/Mistral-7B-Instruct-v0.3' 로 설정함)

```
from transformers import AutoTokenizer, AutoModelForCausalLM

model_dir = '/mnt/nvme01/huggingface/models'
tokenizer = AutoTokenizer.from_pretrained(model_dir + '/MistralAI/Mistral-7B-Instruct-v0.3', trust_remote_code = True)
model = AutoModelForCausalLM.from_pretrained(model_dir + '/MistralAI/Mistral-7B-Instruct-v0.3', trust_remote_code = True)
```

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/datasets 경로

> KorQuAD폴더

파일명	Train Set Size	Validation Set Size	데이터 형식	데이터 용량
KorQuAD/squad_kor_v1	60,407 rows	5,774 rows	CSV	95.3 MB
KorQuAD/squad_kor_v2	83,486 rows	10,165 rows	HF arrows	20.2 GB

*주의: squad_kor_v1 => CSV 파일 형식이라 pandas로 읽고 huggingface datasets으로 변환이 필요함.
'squad_kor_v1'폴더 안에 변환 코드 (conversion.py) 참조할것.

```
from datasets import  Dataset, DatasetDict
import pandas as pd

data_folder = '/mnt/nvme01/huggingface/datasets'

train_df = pd.read_csv(data_folder + '/KorQuAD/squad_kor_v1/train.csv')
val_df   = pd.read_csv(data_folder + '/KorQuAD/squad_kor_v1/validation.csv')
tds      = Dataset.from_pandas(train_df)
vds      = Dataset.from_pandas(val_df)
```

11. 공용 데이터 및 모델 (읽기/쓰기)

/home/shared/huggingface/datasets 경로

> Naver폴더

파일명	Train Set Size	Validation / Test Set Size	데이터 형식	데이터 용량
Naver/nsmc	150,000	50,000	CSV	41.5 MB

*주의: nsmc => CSV 파일 형식이라 pandas로 읽고 huggingface datasets으로 변환이 필요함.
'nsmc'폴더 안에 변환 코드 (conversion.py) 참조할것.

```
from datasets import  Dataset, DatasetDict
import pandas as pd

data_folder = '/mnt/nvme01/huggingface/datasets'

train_df  = pd.read_csv(data_folder + '/Naver/nsmc/train.csv')
test_df    = pd.read_csv(data_folder + '/Naver/nsmc/test.csv')
train_ds  = Dataset.from_pandas(train_df)
test_ds   = Dataset.from_pandas(test_df)
```


Thank
you!